

BRIAN HAYDIN

42.9959° N, 87.8987° W · BAY VIEW, MILWAUKEE

[FIELD NOTES · AGENT SECURITY]

MCP design-review [workbook]

Seven questions, a worked example, and two exercises for reviewing an AI agent and the tools it can call.

From the session **MCP Without Getting Pwned**
The Cloud & AI Summit · October 2, 2026

haydin.ai

github.com/bhaydin/demo-agentops

A worksheet for reviewing one AI agent and the tools it can call. It comes from the session *MCP Without Getting Pwned* (The Cloud & AI Summit, October 2, 2026) and from auditing the demo agent in this repository.

Use it before an agent ships, and again whenever its permissions, tools, or model change. One agent takes about an hour the first time. Put a developer and a security reviewer in the same room.

What's inside

- 1. Seven questions: the review itself, blank
- 2. Tool classification: a table to fill in for every tool
- 3. Registry record: one page per tool server
- 4. Approval check: five yes-or-no questions
- 5. Worked example: all of the above, filled in for the demo agent
- 6. Two exercises: change reviews to run with your team
- 7. When to review again

Three rules for filling it in

Review one agent at a time. Answers that cover "our agents" in general are too vague to act on.

Mark every answer with one of three words.

STATUS	MEANING
OBSERVED	You can point to code, configuration, a test, or a log that shows it.
PROPOSED	You recommend it. It isn't built yet.
TBD	You don't know, or nobody has decided.

A review full of confident answers that nobody verified is worse than one with honest TBDs. The TBDs are the output.

"The prompt tells it not to" is a request, not a control. When a question asks what enforces a limit, write down what stops the action when the model gets it wrong.

[PART 1]

Seven questions

#	QUESTION	WHAT TO WRITE DOWN	YOUR ANSWER	STATUS
1	What task did the user authorize?	The outcome the person asked for. Which actions that allows, and which consequences it rules out. The legitimate workflow you must not break.		
2	Where can untrusted content enter?	Every place the model reads text someone else wrote: user messages, tool results, documents, web pages, tool descriptions and schemas. Who can write to each.		
3	Whose authority does each call use?	The identity and credential on each tool call. What that credential can reach. Who can change it.		
4	What can leave or change?	Each tool's tier (Part 2). Which actions can't be undone. Where data can be sent, and whether output from one server can flow into another.		
5	What enforces each limit?	For every limit in questions 3 and 4: the server-side check, allowlist, validation, or approval that holds when the model is wrong.		
6	How will we establish the outcome?	How you would show, after the fact, what the user asked for, what the agent attempted, and what executed. How long you keep it.		
7	Who owns admission, changes, and response?	A named owner. A named reviewer. How a tool gets approved, how a change gets reviewed, and how access gets turned off.		

How to read your answers

- If question 2 lists a source and question 5 has no enforced limit on what that source can trigger, you have found your most likely incident.
- If question 3 names a person's account, an admin account, or "the key that made the demo work," fix that first.
- If question 7 names a team and not a person, nobody owns it.

[PART 2]

Classify every tool

Sort tools by what happens if one is called when it shouldn't be.

TIER	DEFINITION	DEFAULT HANDLING
Read	Retrieves information. Changes nothing.	Allowed. Check what it can reach and whether its output is trusted.
Write	Changes business state in a way you can undo.	Allowed within the agent's own scope. Validate on the server.
High-impact	Changes state in a way you can't undo, or that costs real money, access, or data.	Held for a human, with the exact action shown.
Admin	Approves actions, grants authority, resets or reconfigures the system.	Not exposed as a tool. A separate path with a separate credential.

Two things teams miss:

- **A Read tool can carry the attack.** If it returns text written by someone else, that text lands in the model's context. Record it in the fourth column.
- **A tool named "search" or "fetch" can be an outbound channel.** If the agent controls the query or the URL, data can leave through it.

TOOL	TIER	WHAT IT CAN REACH	RETURNS TEXT SOMEONE ELSE WROTE?	HELD FOR A HUMAN?	STATUS

[PART 3]

The registry record

One record per tool server. It should fit on a page.

FIELD	YOUR SYSTEM	STATUS
Purpose. What it's for and what it's allowed to change.		
Origin. Publisher, repository, and how you verified it's the real one.		
Owner. A person.		
Version. Three separate things: the source revision, the digest of the deployed artifact, and a snapshot of the tool definitions the model sees.		
Identity and scope. Which credential the agent presents and what it reaches.		
Tool admission. Which tools on this server the agent may use. All of them is an answer, and it should be a deliberate one.		
Approval rule. Which actions wait for a human.		
Visibility. What is recorded per call, and where.		
Change and rollback. Who reviews a change, what evidence they need, and how you undo it.		

A version pin freezes a choice. It doesn't certify that the choice was a good one. A source commit is not a deployed-artifact digest, and neither one is a snapshot of the tool definitions.

[PART 4]

Is your approval step real?

Answer yes or no for each high-impact tool.

#	CHECK	YES / NO
1	Does the approval show the exact action, the exact target, and the identity it will run as? "Allow the agent to continue?" approves nothing.	
2	Is the approval bound to the target that was resolved when the request was made, so it can't apply to something different later?	
3	Is each approval single use?	
4	Are the business preconditions checked again when the approved action runs?	
5	Is approval impossible for the agent to perform? If approval is a tool, the agent can call it, and injected text can tell it to.	

Then two questions about people:

- How many approvals does a normal, legitimate task require? If the answer is more than one or two, expect people to stop reading them.
- Can you tell who approved, not only that someone did?

[PART 5]

Worked example: Swankers Coach

Coach is the demo agent in this repository: a fantasy football assistant built with Microsoft Agent Framework, hosted in Microsoft Foundry, calling a C# MCP server. Reads use real league data. Every write goes to a simulated league. See [ARCHITECTURE.md](#).

The attack it was reviewed against: a rival sends a trade offer whose note field claims league-office authority and tells the assistant to accept the trade and drop a player ([scenario](#)). On October 1, 2026, a deliberately naive version of Coach on a smaller model followed that note on the deployed system. With the owner credential and the approval gate on, both actions were held and then denied, and the league did not change ([evidence](#)).

This example describes the hardened configuration: Coach's own owner credential with the approval gate on.

Seven questions, answered

#	QUESTION	COACH	STATUS
1	What task did the user authorize?	Advice for one franchise. Act only when the owner asks for a specific action.	OBSERVED in the instructions . Gap: an advice-only request doesn't switch the agent to a read-only credential. Nothing on the server knows the request was advice.
2	Where can untrusted content enter?	Trade notes returned by get_trade_offers , written by other owners. User messages. League knowledge files. The tool descriptions the server advertises.	OBSERVED
3	Whose authority does each call use?	Coach's own credential, the owner of franchise 0001. The server derives scope from which credential was presented, not from an argument.	OBSERVED in ScopePolicy.cs . Gap: credentials are shared bearer secrets, not delegated OAuth tokens bound to this server.
4	What can leave or change?	Six read tools, two write tools, two high-impact tools. No tool sends data outside the league.	OBSERVED
5	What enforces each limit?	Server-side scope check. Approval gate on the two high-impact tools. Business validation on writes. Not the prompt.	OBSERVED , with tests. A reviewed per-tool allowlist on the client is PROPOSED . Pinning of tool definitions is PROPOSED .
6	How will we establish the outcome?	Each tool call records the tool, its tier, the caller's scope, the requested and effective franchise, and the gate decision. Approvals are recorded with a confirmation ID. League state is readable.	OBSERVED for traces and state. A named human approver and durable retention are TBD .
7	Who owns admission, changes, and response?	Brian Haydin owns it. Changes to main and to the deployed agent require maintainer approval (REPOSITORY-SECURITY.md).	Owner PROPOSED . Second reviewer TBD .

Tool classification

TOOL	TIER	WHAT IT CAN REACH	RETURNS TEXT SOMEONE ELSE WROTE?	HELD FOR A HUMAN?
get_my_roster	Read	Caller's franchise	No	No
get_roster	Read	Any franchise	No	No
get_player_news	Read	Injury reports	No	No
get_matchup	Read	Caller's matchup	No	No
get_standings	Read	League standings	No	No
get_trade_offers	Read	Offers to the caller	Yes. The note field is free text from the other owner.	No
set_lineup	Write	Caller's franchise	No	No
propose_trade	Write	Caller's franchise	No	No
drop_player	High-impact	Caller's franchise	No	Yes
respond_to_trade	Write to reject, High-impact to accept	Caller's franchise	No	Yes, when accepting

Approve, deny, reset, and seed exist only as REST endpoints behind a separate key ([DemoEndpoints.cs](#)). They are not MCP tools, and a test asserts that the tool list contains no approval tool.

A deliberate trade-off: `set_lineup` and `propose_trade` are not gated. Injected text could still cause a lineup change. That was accepted because the change can be undone and gating it would interrupt the owner every week.

Registry record

FIELD	COACH	STATUS
Purpose	Assist one franchise. Changes the simulated league only.	OBSERVED
Origin	bhaydin/demo-agentops , project Swankers.Mcp	OBSERVED
Owner	Brian Haydin	PROPOSED
Version	Source commit is known. Deployed artifact digest and tool-definition snapshot are not captured as part of admission.	TBD
Identity and scope	Owner credential, one franchise	OBSERVED
Tool admission	The client accepts every tool the server advertises.	Observed gap. A pinned list is Proposed.
Approval rule	Drops and accepted trades wait for a human.	OBSERVED
Visibility	Per-call trace attributes, gate decisions, league state	OBSERVED
Change and rollback	Maintainer review on the repository. Reviewer, decision record, and review date for tool changes are not defined.	TBD

Approval check

#	CHECK	COACH
1	Shows the exact action, target, and identity	Yes. The dialog names the player, the franchise, the tool, and the acting scope.
2	Bound to the target resolved at request time	Yes. Test: Approval_is_bound_to_the_target_resolved_at_confirmation_time
3	Single use	Yes, per confirmation. Test: Approve_executes_the_parked_action_exactly_once . Repeated tool calls create new confirmations; there is no deduplication.
4	Preconditions rechecked at execution	Yes
5	The agent cannot approve	Yes. Test: Tool_list_matches_the_architecture_and_has_no_approval_tool

Tests are in [McpServerTests.cs](#) and [ConfirmationGateTests.cs](#).

Open items from this review

GAP	WHAT PRODUCTION NEEDS
Shared bearer secrets	Delegated OAuth, with tokens issued for this server and never passed through to an upstream service
One shared key approves actions	A named human approver
Pending approvals held in memory, with no expiry	Durable records that time out
Client accepts every advertised tool	A reviewed, pinned tool list and a check for changed definitions
No outbound tool, so data flow was never tested	Destination limits and a review of which read tools can feed which send tools

[PART 6]

Two exercises

Run these with developers and security reviewers together. Each takes about twenty minutes. The scenarios use Coach. Substitute one of your own tools once the pattern is clear.

Exercise 1. Fewer interruptions, more authority

The request. A developer reports that the approval dialogs slow the team down. The proposal: keep the same tools and the same instructions, switch the agent from the owner credential to the commissioner credential, and turn the approval gate off.

Fill in

FIELD	YOUR ANSWER
Business need for the change	
Resources newly in reach	
Actions that lose their confirmation	
What an attacker gains if the model is fooled	
Reviewer	
Decision	
Evidence required	
Expiry, and how it gets rolled back	

What a good review notices

- Nothing in the tool list changed. The tool names, schemas, and instructions are identical, and the risk is completely different. A credential or configuration change is a security change.
- The proposal removes two separate controls. Scope decides whose resources the agent can reach. Approval decides what happens inside that reach. Evidence should show both: a cross-franchise write denied by scope, and an own-franchise high-impact action held by the gate.
- The complaint is real. Before refusing, count how many dialogs a legitimate task produces. If routine work triggers approvals, fix which tools are gated. Don't widen the credential.

Suggested outcome. Keep the narrow credential and the gate. If friction is the problem, review the tiering in Part 2.

Exercise 2. A familiar tool with a changed definition

The request. This is hypothetical; no such change exists in this repository. A new build of the tool server keeps the tool `respond_to_trade`. Its description has been reworted, and the `accept` parameter, which was required, is now optional with a default.

Fill in

FIELD	YOUR ANSWER
What changed, line by line: description, parameters, defaults, schemas, annotations	
What a call with the argument omitted does now	
Effect on scope and on the approval gate	
Whether the deployed artifact matches what was reviewed	
Tests or evidence required	
Reviewer	
Decision	

What a good review notices

- The name didn't change, so a review that checks tool names passes it.
- Before, the model had to state an intention. Now a call with no argument does something. If the default is to accept, an agent that probes the tool has accepted a trade. No prompt injection is involved.
- Annotations such as read-only or destructive describe behavior. They enforce nothing, and annotations from a server you don't control are untrusted.
- The question to answer from the code, not from the description: which branch does an omitted argument take, and is that branch still gated?

Suggested outcome. Hold admission until the changed behavior is reviewed and tested. Compare the full definition, and the deployed artifact, against what was approved.

[PART 7]

When to review again

Repeat the seven questions when any of these change:

- The credential, identity, or scope the agent uses
- The tool list, or any tool's description, parameters, or defaults
- The tool server's version or its dependencies
- The model or the instructions. A model change is a version change; injection resistance varies between models and nothing in your code will tell you.
- Where data can be sent
- Who owns or reviews it

Close the review

Decision	
Named approver	
Open items and their owners	
Approved version and evidence	
Next review date or trigger	

References

- [OWASP MCP Security Cheat Sheet](#): least privilege, tool description and schema integrity, human review of sensitive actions, supply chain, and prompt injection through tool return values.
- [MCP specification, authorization security considerations](#): tokens must be issued for the receiving server, and a server must not pass a client's token through to an upstream API.
- Demo system and evidence in this repository: [architecture](#), [presenter runbook](#), [deployed containment evidence](#).

The seven questions are a practical review method derived from auditing this agent. They are not taken from an external standard.